INTELLI CODE ANALYZER

AI Technical Debt Analysis & Optimization Comparison Report

Original Code Analysis | AI-Powered Refactoring | Before & After Results

Key Performance Snapshot

StudentManagementSystem (PRJ-2026-001) · Java / Maven · Optimization Comparison · Sample results for layout reference

72 Debt Score (Before)	**48** Debt Score (After)	**33.33%** Improvement	**−26** Issues Reduced
64 → 38 Total Issues	**6** Files Optimized	**97.62%** Parse Success	**50.62 s** Total Workflow

Supporting scan metrics

42 Files discovered	**40 → 41** Parsed (before → after)	**2 → 1** Parse failures	**1.8 → 1.7 MB** Project size
3,240 → 3,190 Lines of code	**8.4 s** Original scan time	**42.6 s** AI generation time	**7.9 s** Optimized rescan
4 → 1 Critical issues	**12 → 7** High issues	**51 → 29** Code smells	**3 → 2** Security findings

Note: All names, dates, measurements and results in this sample are illustrative. The production report must populate them from actual scan and optimization outputs. Debt-score improvement and issue-count reduction are separate measures.

Report Overview

This report was generated by the Intelli Code Analyzer for the project StudentManagementSystem (identifier: PRJ-2026-001), a Java/Maven application. It is an Optimization Comparison report, produced in three stages: the original source project was scanned on 03-Nov-2026 at 10:15, the AI-assisted optimization was completed at 10:22, and the optimized project was rescanned with the final report generated at 10:25.

1. Executive Summary

This report summarizes the technical debt detected in the original project, the AI-assisted refactoring performed on selected issues, and the results of rescanning the optimized project. The original source project is preserved; changes are written to a separate optimized copy.

The headline result is a debt-score improvement of 33.33% (from 72 to 48) and an issue-count reduction of 40.63% (from 64 to 38 issues). The outcome is assessed as partially successful: the optimized scan shows a lower debt score and fewer findings, while some issues remain and two new findings were detected. The new and remaining findings should be reviewed before accepting the optimized project.

Key Metric	Previous Code	Current Code	Change

Technical debt score (0–100; lower is better)	72	48	−24 points
Total detected issues	64	38	−26 issues
Critical issues	4	1	−3
High issues	12	7	−5
Medium issues	21	14	−7
Low issues	27	16	−11
Code smells	51	29	−22
Security findings	3	2	−1
Files modified by AI	—	6	6 files

2. Project Overview

The original project, StudentManagementSystem, and its AI-generated optimized copy, StudentManagementSystem_Optimized, are both Java/Maven projects built from a pom.xml configuration. Both contain 18 classes, 3 interfaces and 126 methods, so the refactoring did not alter the overall code structure. Parsing was slightly better in the optimized copy: 41 of 42 discovered files parsed successfully, versus 40 of 42 in the original, reducing parse failures from 2 to 1. The optimized project is also marginally smaller (1.7 MB vs. 1.8 MB).

Project path and scan identifiers should be retained internally for traceability. Avoid exposing sensitive absolute server paths in a user-facing report.

Project Attribute	Original Project	Optimized Project
Project name	StudentManagementSystem	StudentManagementSystem_Optimized
Build configuration	pom.xml	pom.xml
Files discovered / parsed	42 / 40	42 / 41
Files that failed parsing	2	1
Project size	1.8 MB	1.7 MB
Classes / Interfaces	18 / 3	18 / 3
Methods	126	126

3. Codebase Statistics

The optimization touched 6 files, adding 38 lines and removing 88 lines. Total lines of code dropped from 3,240 to 3,190, while comment lines actually increased (310 → 325), suggesting the AI added explanatory comments while simplifying code. Blank lines fell slightly (420 → 415).

File size and line-count reductions are descriptive only; smaller code is not automatically better. Interpret them together with behavior preservation, tests and quality metrics.

Statistic	Previous Code	Current Code
Files parsed	40	41
Lines of code	3,240	3,190
Comment lines	310	325
Blank lines	420	415
Classes / Interfaces / Methods	18 / 3 / 126	18 / 3 / 126
Total project size	1.8 MB	1.7 MB
Files modified	—	6
Lines added / removed	—	+38 / −88

4. Code Smell Analysis

Code smells are maintainability warning signs reported by the configured static-analysis rules. Almost every smell category improved after optimization: Long Method fell from 8 to 3 occurrences, God Class from 2 to 1, Duplicate Code from 12 to 5, Unused Variables from 14 to 8, and High Cyclomatic Complexity from 6 to 4. Magic Numbers improved only partially (9 → 7), and Excessive Parameters (4 → 3) is still present.

Code Smell / Rule	Severity	Before	After	Status
Long Method	High	8	3	Improved
God Class	High	2	1	Improved
Duplicate Code	Medium	12	5	Improved
Unused Variable	Low	14	8	Improved
Magic Number	Low	9	7	Partially improved
High Cyclomatic Complexity	High	6	4	Improved
Excessive Parameters	Medium	4	3	Still present

4.1 Representative Issue Example

As a representative example, the LONG_METHOD rule (severity: High, category: Maintainability) flagged the registerStudent method of StudentService in src/main/java/service/StudentService.java at line 84. The scanner reported that the method contained multiple responsibilities and exceeded the configured complexity threshold. The AI refactored it into smaller, focused private methods that extract cohesive validation and persistence operations, while retaining the method's behavior and public signature. The rescan confirmed the finding count was reduced, though tests still need to be verified.

5. Technical Debt Analysis

The total issue count fell from 64 to 38 (−26 issues), with reductions across every severity level — the largest gains coming from Low (−11) and Medium (−7) issues. Only 1 critical issue remains, down from 4.

The debt score improved from 72 to 48, a delta of −24. Improvement percentage = ((72 − 48) / 72) × 100 = 33.33%. Confirm the configured score direction before interpreting this metric.

Severity	Before	After	Delta (After − Before)
Critical	4	1	−3
High	12	7	−5
Medium	21	14	−7
Low	27	16	−11
TOTAL	64	38	−26

6. Code Quality and Maintainability

Every quality metric moved in the right direction in this sample. Average cyclomatic complexity dropped from 7.2 to 5.8, and the maximum fell sharply from 24 to 16. Only 4 methods remain above the configured complexity threshold, down from 6. Class coupling improved from 0.68 to 0.55, and code duplication was nearly halved, from 8.5% to 5.1%.

Only include metrics actually calculated by the scanner. If a metric is unavailable, show N/A rather than estimating it.

Metric	Before	After	Interpretation
Average cyclomatic complexity	7.2	5.8	Lower in sample
Maximum cyclomatic complexity	24	16	Lower in sample
Methods above threshold	6	4	Reduced
Class coupling (scanner metric)	0.68	0.55	Lower; confirm metric definition
Code duplication	8.5%	5.1%	Reduced

7. Dependency Analysis

The project uses 15 dependencies, and the optimization did not change any of them: 2 remain outdated and 1 remains vulnerable in both the baseline and optimized scans. Key packages such as spring-core, jackson-databind and junit were unchanged. They must be verified against actual scan data before any conclusion is drawn.

Dependency Metric	Before	After
Total dependencies	15	15
Outdated dependencies	2	2
Vulnerable dependencies	1	1
Dependency changes	—	0

8. Security Analysis

Source-code security findings dropped from 2 to 1: the Medium-severity finding was resolved, while the single High-severity finding remains. No critical or low source findings were detected in either scan. Separately, one third-party dependency vulnerability persists in both scans and is covered by the dependency analysis workflow.

Security Measure	Before	After
Total source-code security findings	2	1
Critical	0	0
High	1	1
Medium	1	0
Low	0	0
Dependency vulnerabilities	1	1

9. AI Optimization Summary

The AI optimization workflow selected 12 issues and processed 6 files. All 6 files were modified successfully, with 1 issue failing generation. AI generation took 42.6 seconds, after which the optimized project was rescanned successfully. Record the actual model/provider and timings from execution logs in production reports.

Optimization Metric	Sample Result
AI provider / model	Configured provider / configured model
Issues selected	12
Files processed / modified / generated	6 / 6 / 6
Failed issues	1
AI generation duration	42.6 seconds
Optimized project rescan	Completed

File	Issue Addressed	Result
StudentService.java	Long Method; Duplicate Code	Optimized
CourseManager.java	High Complexity	Optimized
Student.java	Unused Variable	Optimized
ReportService.java	God Class	Partially optimized
ValidationUtil.java	Duplicate Code	Optimized
EnrollmentService.java	Excessive Parameters	Failed

10. Before and After Issue Comparison

Issues are matched across scans using rule, class, method and file identity. Fixed means a baseline finding was no longer detected after rescan; New means a finding appeared only in the optimized scan; Still present means the matched finding remains. A severity change alone does not mean an issue is fixed.

Comparison Result	Count	Definition
Fixed	28	Baseline issues no longer detected after rescan
New	2	Findings detected in current scan but not baseline
Still present	10	Matched findings remain
Improved	5	Measured characteristics improved, but finding remains

10.1 Fixed Issues (sample)

Rule	File	Severity Before	Status
DUPLICATE_CODE	StudentService.java	Medium	Fixed
UNUSED_VARIABLE	Student.java	Low	Fixed

10.2 New Issues (sample)

Rule	File	Severity	Recommendation

NEW_COMPLEXITY_WARNING	ReportService.java	Medium	Review the modified control flow
STYLE_WARNING	CourseManager.java	Low	Apply the configured formatting convention

10.3 Still-Present Issues (sample)

Rule	File	Severity	Next Action
LONG_METHOD	ReportService.java	High	Split remaining responsibilities
OUTDATED_DEPENDENCY	pom.xml	Medium	Review dependency update separately

11. Analysis and Optimization Efficiency

Original scan completed in 8.4 seconds with a parse success rate of 95.24%. The optimized scan completed in 7.9 seconds with 97.62% parse success. AI generation accounted for most of the workflow time (42.6 s), and comparison completed in 0.12 s. Total optimization workflow time was approximately 50.62 seconds.

Scan duration is analyzer performance, not proof that the target application runs faster.

Performance Metric	Original Scan	Optimized Scan / Workflow
Scan duration	8.4 s	7.9 s
Files discovered	42	42
Files parsed	40	41
Parse success rate	95.24%	97.62%
AI generation duration	—	42.6 s
Comparison duration	—	0.12 s
Total optimization workflow	—	50.62 s

12. File-Level Change Summary

Six files were modified by the AI optimizer. The table below summarizes size and issue-count changes for representative files. Include a full file-level table in the generated production report.

File	Size Before	Size After	Issues Before	Issues After	Result
StudentService.java	12.4 KB	11.8 KB	8	3	Improved
CourseManager.java	8.2 KB	8.5 KB	5	2	Improved

Student.java	4.1 KB	3.9 KB	3	1	Improved

13. Class and Method Analysis

Class-level issue counts also improved for several key components. StudentService and CourseManager showed clear reductions, while ReportService still carries a significant remaining load and should be prioritized next.

Class / Method	Before	After	Status
StudentService	8 findings	3 findings	Improved
CourseManager	5 findings	2 findings	Improved
ReportService	6 findings	5 findings	Still present

14. Reliability and Detection Coverage

Parse coverage improved slightly after optimization. Precision, recall and F1 are marked N/A because they require labeled ground-truth findings and must not be inferred from parser success alone.

Measure	Original	Optimized
Discovered files	42	42
Parsed files	40	41
Parse success rate	95.24%	97.62%
Parse failures	2	1
Precision / recall / F1	N/A	N/A

15. Recommendations

- Prioritize the remaining high-severity Long Method finding in ReportService.java.
- Review the newly detected findings in the optimized scan before accepting the generated project.
- Assess outdated and vulnerable dependencies through the dependency analysis workflow.
- Run the project’s unit and integration tests to validate behavior preservation.
- Review AI-generated diffs before merging or deploying the optimized project.

16. Final Summary

In this illustrative example, the optimized project has a lower technical debt score and fewer detected issues than the baseline. Several code smells were reduced, while some findings remain and new findings require review. The AI-generated project should be reviewed and tested before adoption.

Final Result	Value

Debt score change	72 → 48
Debt score improvement	33.33%
Issue count change	64 → 38
Issues fixed	28
New issues	2
Still present	10
Files modified	6
Optimization status	Partially successful — review required

17. Detailed Issue Appendix (Sample)

The production report should include every issue from both scans, with pagination and repeated table headers. Appendix issue records should also include class, method, message, suggestion, baseline/current details, and comparison status.

ID	Rule	Severity	File / Location	Status
ISS-001	LONG_METHOD	High	StudentService.java:84	Improved
ISS-002	DUPLICATE_CODE	Medium	StudentService.java:120	Fixed
ISS-003	UNUSED_VARIABLE	Low	Student.java:31	Fixed
ISS-004	OUTDATED_DEPENDENCY	Medium	pom.xml	Still present

18. Report Definitions and Notes

- Previous code means the original uploaded project and its baseline scan.
- Current code means the separately generated optimized project and its post-optimization scan.
- Debt score delta = current score − baseline score.
- Debt score improvement (%) = (baseline score − current score) / baseline score × 100, when baseline score > 0.
- Issue count delta = current issue count − baseline issue count.
- Fixed means a baseline finding was not detected in the current scan under the configured matching method; it does not independently prove functional correctness.
- N/A means the metric was not supplied or could not be calculated.
- AI-generated changes must be reviewed and tested. This report is not a substitute for tests, code review, or security verification.